Vehicle Parking Management System (VPMS)

CSC2104: Object Oriented Programming in Java • Comprehensive Viva & Code Reference Guide

Java 21 LTS | Spring Boot 3.3.4 | MySQL 8.0 | React 18 + Tailwind | Full OOP Syllabus

📍 **Code Directory on Disk:**

`C:\Users\Asus\Desktop\Vehicle-Parking-Management-System\backend\src\main\java\com\parking\management\`

PART 1: THE FOUR CORE OOP PILLARS

1. Abstraction

`model/Vehicle.java (Lines 25–88)`

Concept: Hiding internal implementation complexity and enforcing an abstract contract that all subclasses must satisfy.

```
public abstract class Vehicle implements ParkingFeeCalculator, PrintableReceipt {
    // Abstract method — no generic formula exists; subclasses must provide it!
    public abstract double calculateParkingFee(long durationHours);
    public abstract String getVehicleType();
}
```

🗣️ **WHAT TO SAY TO PROFESSOR:**

“Sir, a Vehicle is an abstract concept. You cannot park a generic vehicle without knowing whether it is a Bike, Car, or Truck because each has distinct physical footprint and tariff rates. Thus, `Vehicle` is an `abstract class` with abstract method `calculateParkingFee()` enforced on subclasses.”

2. Encapsulation

`model/Vehicle.java & model/ParkingSlot.java`

Concept: Restricting direct field access using `private` keywords and protecting state through validated getters and setters.

```
private String vehicleNumber;
private String ownerName;

public void setVehicleNumber(String vehicleNumber) {
    if (vehicleNumber == null || vehicleNumber.trim().isEmpty()) {
        throw new IllegalArgumentException("License number cannot be blank");
    }
    this.vehicleNumber = vehicleNumber.trim().toUpperCase();
}
```

WHAT TO SAY TO PROFESSOR:

"Sir, all model attributes are declared `private` to prevent state corruption. For instance, external classes cannot set a null plate number or invalid slot; setters enforce validation and defensive sanitization."

3. Inheritance (Multilevel & Hierarchical)

`model/TwoWheeler.java, FourWheeler.java, Bike/Car/Truck.java`

Concept: Reusing common attributes/methods across parent and child classes using `extends` and the `super` keyword.

- **Multilevel:** `Vehicle` → `TwoWheeler` → `Bike`
- **Hierarchical:** `Vehicle` → `FourWheeler` → branched into `Car` and `Truck`
- **super keyword:** `super(vehicleNumber, ownerName, phoneNumber);` delegates common initialization to parent constructor.

WHAT TO SAY TO PROFESSOR:

"Sir, we use a 3-tier inheritance tree. Common attributes like registration number are in `Vehicle`. Category attributes like `numberOfAxles` are in `FourWheeler`. Specific types like `Car` and `Truck` inherit and reuse parent logic via `super()`."

4. Polymorphism (Compile-Time & Runtime)

`Vehicle.java & service/ParkingService.java (Lines 162-168)`

A. Compile-Time (Method Overloading): Same method name with different arguments.

```
// Vehicle.java
double calculateParkingFee(long durationHours);                        // Standard rate
double calculateParkingFee(long durationHours, double discountPercent); // Overload with discount
double calculateParkingFee(Integer hoursWrapper);                      // Overload with Autoboxing
```

B. Runtime (Dynamic Method Dispatch / Overriding):

```
// ParkingService.java (Line 164)
Vehicle vehicle = session.getVehicle();                       // Reference is abstract Vehicle
double parkingFee = vehicle.calculateParkingFee(durationHours); // Dynamic dispatch to Bike/Car/Truck!
```

WHAT TO SAY TO PROFESSOR:

"Sir, in `ParkingService.checkoutVehicle`, the variable reference is of type `Vehicle`. We do not use clumsy `if-else` checks. At runtime, the JVM inspects the heap object and dynamically invokes the correct tariff method (Car: ₹30 1st hr, Bike: ₹20 1st hr, Truck: ₹50 1st hr)."

PART 2: ADVANCED SYLLABUS CONCEPTS

5. Interfaces & Default Methods

`interfaces/ParkingFeeCalculator.java & PrintableReceipt.java`

Java eliminates the Diamond Problem by avoiding multiple class inheritance, but achieves multiple inheritance of type via interfaces.

```
public interface PrintableReceipt {
    String formatReceiptDetails(String slotNumber, long durationHours, double fee, String paymentMethod);

    // Java 8+ Default Method: shared receipt footer across all subclasses
    default String getReceiptFooter() {
        return "========================================\n" +
               "  Thank you for using VPMS Parking!     \n" +
               "========================================";
    }
}
```

6. Multidimensional Array (2D Grid) & Recursion

`model/ParkingLot.java (Lines 28–118)`

Physical facility is modeled as a 2D matrix: `ParkingSlot[floorIndex][slotIndex]`. Finding a free slot uses recursive grid search:

```
private ParkingSlot searchGridRecursively(SlotType slotType, int floorIndex, int slotIndex) {
    if (floorIndex >= totalFloors) return null; // Base Case 1: All floors searched
    if (slotIndex >= slotsPerFloor) return searchGridRecursively(slotType, floorIndex + 1, 0);

    ParkingSlot slot = parkingGrid[floorIndex][slotIndex];
    if (slot != null && slot.isAvailable() && slot.getSlotType() == slotType) {
        return slot; // Base Case 2: Free matching slot found!
    }
    return searchGridRecursively(slotType, floorIndex, slotIndex + 1); // Recursive Step
}
```

7. Nested Classes (Static Nested vs Inner)

`model/ParkingLot.java (Lines 244–340)`

- **Static Nested Class (`ParkingLot.ParkingStatistics`):** Independent analytical helper class. Can be instantiated without creating an outer `ParkingLot` instance.
- **Non-Static Inner Class (`ParkingLot.SlotManager`):** Tightly coupled to an enclosing `ParkingLot` instance with direct access to private `parkingGrid`.

8. Multithreading, Synchronization & wait() / notifyAll()

`threads/ParkingMonitor.java, ReportGenerator.java, ParkingRequestQueue.java`

- **Extending Thread:** `ParkingMonitor extends Thread` with `setDaemon(true)` and safe shutdown via `volatile boolean running`.
- **Implementing Runnable:** `ReportGenerator implements Runnable` + `thread.join()`.
- **Synchronized Block:** `synchronized(slotAllocationLock) { assignedSlot.occupy(); }` prevents race conditions when concurrent booking requests arrive.
- **wait() & notifyAll():** Inside `ParkingRequestQueue`, waiting threads call `wait()` until a vehicle leaves and triggers `notifyAll()`.

9. File I/O Streams (Character vs Byte Streams)

`io/ParkingReportWriter.java & io/ParkingBackupManager.java`

- **Character Streams:** `FileWriter` + `BufferedWriter` + `try-with-resources` to export human-readable `parking_history.txt` reports.
- **Byte Streams (Binary):** `FileOutputStream` + `BufferedOutputStream` to serialize raw binary snapshots into `parking_backup.dat`.

PART 3: COMPLETE VIVA CODE MAP

Concept	Java File Location	Key Implementation Highlight
Abstraction	`model/Vehicle.java`	`public abstract class Vehicle` with abstract fee method
Encapsulation	`model/Vehicle.java`, `ParkingSlot.java`	Private fields, validated getters/setters
Multilevel Inheritance	`Vehicle → TwoWheeler → Bike`	3-tier tree reusing code via `super()`
Hierarchical Inheritance	`Vehicle → FourWheeler → Car, Truck`	Branched subclasses sharing four-wheeler attributes
Runtime Polymorphism	`service/ParkingService.java#L164`	Dynamic dispatch calling overridden fee logic
Method Overloading	`Vehicle.java`, `ParkingLot.java`	Overloaded fee calculation & slot search signatures
Interfaces & Defaults	`interfaces/PrintableReceipt.java`	Interface contract with Java 8 `default` footer
2D Array Grid	`model/ParkingLot.java#L28`	`ParkingSlot[totalFloors][slotsPerFloor]`
Recursion	`model/ParkingLot.java#L88-118`	`searchGridRecursively()` with base cases
Nested & Inner Classes	`model/ParkingLot.java#L244-340`	Static `ParkingStatistics` vs Inner `SlotManager`
Thread vs Runnable	`threads/ParkingMonitor` & `ReportGenerator`	`Thread` extension vs `Runnable` with `join()`
Synchronization	`service/ParkingService.java#L93`	`synchronized(lock)` prevents concurrent race conditions
wait() & notifyAll()	`threads/ParkingRequestQueue.java`	Thread suspension and event wakeup signaling
File Handling (Char/Byte)	`io/ParkingReportWriter` & `BackupManager`	`FileWriter` text vs `FileOutputStream` binary
Checked Exceptions	`exception/ParkingLotFullException.java`	Subclasses `Exception` requiring `throws` clause

PART 4: TOP 5 VIVA QUESTIONS & ANSWERS

Q1. Is Java Pass-by-Reference or Pass-by-Value?

Answer: Java is strictly **Pass-by-Value** at all times. When passing primitives, a copy of the value is passed. When passing object references, a copy of the reference address pointer is passed by value. Reassigning the pointer inside the method doesn't change the caller's reference, but mutating properties through that pointer does. (Demonstrated in `demo/ArgumentPassingDemo.java`).

Q2. What is the difference between Thread's start() and run()?

Answer: Calling `run()` executes the code synchronously in the current thread like a standard method call. Calling `start()` instructs the JVM and OS to allocate a new call stack and launch a distinct background thread of execution. (Demonstrated in `threads/ParkingMonitor.java`).

Q3. Why use try-with-resources instead of explicit finally blocks?

Answer: Any class implementing `AutoCloseable` (e.g. `FileWriter` or `FileOutputStream`) is guaranteed to close cleanly upon block exit, even if unexpected exceptions occur, preventing file lock and memory leaks. (Demonstrated in `io/ParkingReportWriter.java`).

Q4. Can an Abstract Class have a constructor?

Answer: Yes! `Vehicle` has constructors. While you cannot instantiate it directly via `new Vehicle()`, its constructor is invoked by subclass constructors via `super()` to initialize common properties like vehicle number and timestamp.

Q5. Why is the payment collection on the Admin/Operator screen?

Answer: In real parking facilities, drivers do not access database systems directly. The exit boom barrier has an operator booth terminal where the attendant verifies the vehicle, states the calculated tariff, collects payment (Cash/UPI/Card), records the transaction in MySQL, prints the tax receipt, and releases the slot.
